

In []:

```
# Initialize Otter
import otter
grader = otter.Notebook("lab02_.ipynb")
```

 Open in Colab

(https://colab.research.google.com/github/develop_otter/material_20242/blob/main/lab/lab01/lab01_.ipynb)

In []:

```
import otter
import pandas as pd
import numpy as np
%matplotlib inline
import otter
import matplotlib.pyplot as plt
from datascience import *
grader = otter.Notebook()
path="https://raw.githubusercontent.com/fundatosudea/material_20242/develop_otter"
```



UNIVERSIDAD
DE ANTIOQUIA
1 8 0 3

Fundamentos en ciencias de datos

Laboratorio 2: Operaciones con tablas

¡Bienvenidos al Laboratorio 2! Esta semana, aprenderemos cómo importar un módulo y practicaremos operaciones con tablas. Algunas referencias están dadas en el siguiente enlace [enlace](https://www.data8.org/fa23/reference/) (<https://www.data8.org/fa23/reference/>), el cual tiene información útil para esta práctica de laboratorio.

Lectura recomendada:

- [Introducción a las Tablas](https://www.inferentialthinking.com/chapters/03/4/Introduction%20to%20Tables) (<https://www.inferentialthinking.com/chapters/03/4/Introduction to Tables>)

Como recordatorio, estas son las políticas para obtener una nota completa:

1. Para los estudiantes inscritos en el **Laboratorio regular** en persona, recibirán crédito de laboratorio al **asistir** a la sección de laboratorio y lograr un progreso significativo en `notebook` de `jupyter`.

Envío: Una vez que haya terminado, ejecute todas las celdas además de la última, seleccione Archivo > Guardar cuaderno y luego ejecute la celda final. El resultado contendrá un archivo zip que podrá utilizar para enviarlo.

Comencemos configurando las pruebas e importaciones ejecutando la siguiente celda.

1. Revisión: Los componentes básicos del código Python

Los dos bloques de construcción del código Python son *expresiones* y *declaraciones*. Una **expresión** es un fragmento de código que:

- Es autónomo, lo que significa que tendría sentido escribirlo en una línea por sí solo, y
- generalmente se evalúa como un valor.

Aquí hay dos expresiones que se evalúan como 3:

```
3
5 - 2
```

Un tipo importante de expresión es: `call expression` en inglés o **expresión de ejecución**. Una expresión de ejecución se utiliza para describir la acción de llamar o invocar una función o método en programación, comienza con el nombre de una función y va seguida de los argumentos de esa función entre paréntesis. La función devuelve algún valor, según sus argumentos. Algunas funciones matemáticas importantes se enumeran a continuación.

Función	Descripción
<code>abs</code>	Devuelve el valor absoluto de su argumento

^{max} Función	Devuelve el máximo de todos sus argumentos
	Descripción
min	Devuelve el mínimo de todos sus argumentos
pow	Eleva su primer argumento a la potencia de su segundo argumento
round	Redondea su argumento al entero más cercano

Aquí hay dos expresiones de ejecución que se evalúan como 3:

```
abs(2 - 5)
max(round(2.8), min(pow(2, 10), -1 * pow(2, 10)))
```

La expresión `2 - 5` y las dos llamadas de ejecuciones anteriores, son ejemplos de **expresiones compuestas**, lo que significa que en realidad son combinaciones de varias expresiones más pequeñas. `2 - 5` combina las expresiones `2` y `5` mediante resta. En este caso, `2` y `5` se denominan **subexpresiones** porque son expresiones que forman parte de una expresión más grande.

Una **declaración** es una línea completa de código. Algunas declaraciones son sólo expresiones. Las expresiones enumeradas anteriormente son ejemplos.

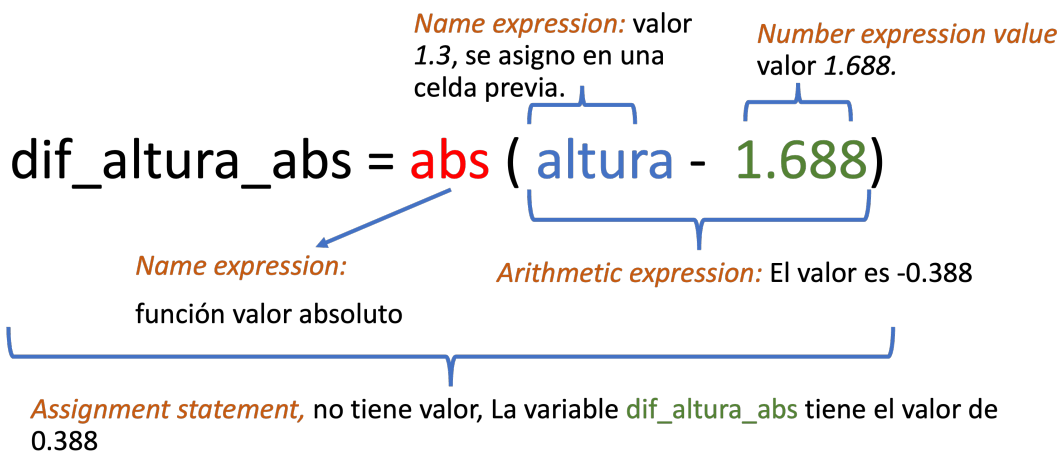
Otras declaraciones *hacen que algo suceda* en lugar de *tener un valor*. Por ejemplo, una **declaración de asignación** asigna un valor a un nombre.

Una buena forma de pensar en esto es que estamos **evaluando el lado derecho** del signo igual y **asignándolo al lado izquierdo**. Aquí hay algunas declaraciones de asignación:

```
altura = 1.3
num_cinco = abs(-5)
dif_altura_abs = abs(altura - 1.688)
```

Una idea importante en programación es que se pueden construir cosas grandes e interesantes combinando muchas cosas simples y poco interesantes. La clave para comprender un código complicado es **dividirlo en sus componentes simples**

Por ejemplo, suceden muchas operaciones en la última declaración como combinación operaciones más básicas. La siguiente imagen describe mejor lo que está pasando (Notese que algunos nombres están en inglés, para ir familiarizandose con algunos nombres que son de interés en python)



Pregunta 1.1. En la siguiente celda, asigne el nombre `nuevo_año` al **número mayor** entre los dos números siguientes:

- el **valor absoluto** de $2^6 - 2^{11} - 2^5 - 8$, y
- $5 \times 13 \times 31 + 7$.

Intente utilizar solo una declaración (una línea de código). Asegúrese de verificar su trabajo ejecutando la celda de prueba después.

In []:

```
# BEGIN SOLUTION NO PROMPT
nuevo_año = max (abs (2**6-2**11-2**5-8),      5*13*31+7)
# END SOLUTION

""" # BEGIN PROMPT
nuevo_año=...
""" # END PROMPT
```

In []:

```
grader.check("q11")
```

En el ejemplo anterior, creamos una sola línea de código porque solo involucra operaciones matemáticas. Sin embargo, las preguntas de programación más complicadas requerirán más pasos. No siempre es una buena idea agrupar estos pasos en una sola línea porque puede hacer que el código sea más difícil de leer y de depurar.

Una buena práctica de programación implica dividir el código en pasos más pequeños y utilizar nombres apropiados. ¡Tendrás mucha práctica en el resto de este curso!

2. Importación de código

La mayor parte de la programación implica un trabajo muy similar al trabajo que se ha realizado antes. Dado que escribir código lleva mucho tiempo, es bueno confiar en el código publicado por otros cuando sea posible. En lugar de copiar y pegar, Python nos permite **importar módulos**. Un módulo es un archivo con código Python que tiene variables y funciones definidas. Al importar un módulo, podemos usar su código en nuestro propio cuaderno.

Python incluye muchos módulos útiles que están a sólo una "importación" de distancia. Veremos el módulo "math" como primer ejemplo. El módulo "math" es extremadamente útil para calcular expresiones matemáticas en Python.

Supongamos que queremos calcular con mucha precisión el área de un círculo A con un radio R de 5 metros. Para eso, necesitamos la constante π , que es aproximadamente 3.14. Convenientemente, el módulo "math" tiene "pi" definido para nosotros. Ejecute la siguiente celda para importar el módulo matemático:

$$A = \pi R^2 = \pi 5^2 \text{ m}^2$$

In []:

```
import math
radio = 5
area_del_circulo = radio**2 * math.pi
area_del_circulo
```

En el código anterior, la línea "import math" importa el módulo matemático. Esta declaración crea un módulo y luego asigna el nombre "math" a ese módulo. Ahora podemos acceder a cualquier variable o función definida en `math` escribiendo el nombre del módulo seguido de un punto, luego seguido del nombre de la variable o función que queremos.

<nombre del módulo>.<nombre>

Pregunta 2.1. El módulo `math` también proporciona el nombre `e` para la base del logaritmo natural, que es aproximadamente 2.71. Calcule $e^\pi - \pi$, dándole el nombre `cerca_veinte`.

Recuerda: ¡También puedes acceder a `pi` desde el módulo `math`!

In []:

```
from math import *
# BEGIN SOLUTION NO PROMPT
cerca_veinte = e**math.pi-math.pi
# END SOLUTION
""" # BEGIN PROMPT
cerca_veinte=...
""" # END PROMPT
cerca_veinte
```

In []:

```
grader.check("q21")
```

2.1. Accediendo a funciones

En la pregunta anterior, accedí a variables dentro del módulo "math".

Los módulos también definen **funciones**. Por ejemplo, "math" proporciona la función "floor" que se utiliza en programación para redondear un número hacia abajo al entero más cercano. Habiendo importado ya "math", podemos escribir "math.floor(7.5)" para calcular el `floor` de 7.5 que es 7.

Pregunta 2.1.1. Calcule el `floor` del número π usando el módulo `math`. Asigne al resultado el nombre `floor_de_pi`.

In []:

```
from math import *
# BEGIN SOLUTION NO PROMPT
floor_de_pi=floor(pi)
# END SOLUTION
""" # BEGIN PROMPT
floor_de_pi=...
""" # END PROMPT
floor_de_pi
```

In []:

```
grader.check("q211")
```

Para que tengas como referencia, a continuación se muestran algunos ejemplos más de funciones del módulo "math".

Observe cómo diferentes funciones toman diferentes números de argumentos. A menudo, la [documentación](https://docs.python.org/3/library/math.html) (<https://docs.python.org/3/library/math.html>) del módulo proporcionará información sobre cuántos argumentos se requieren para cada función.

Sugerencia: si presiona `shift+tab` mientras está al lado de la llamada de función, aparecerá la documentación para esa función.

In []:

```
# Calcular logaritmos (el logaritmo de 8 en base 2).
# El resultado es 3 porque 2 elevado a 3 es 8.
math.log(8, 2)
```

In []:

```
# Calcular raíces cuadradas.
math.sqrt(5)
```

Hay varias formas de importar y acceder a código desde fuentes externas. El método que usamos anteriormente, `<importar>`, `<nombre_módulo>`, importa el módulo completo, requiere que usemos `<nombre_módulo>.<nombre>` para acceder a su código.

También podemos importar una constante o función específica en lugar del módulo completo. Tenga en cuenta que no es necesario utilizar el nombre del módulo de antemano para hacer referencia a ese valor en particular. Sin embargo, debe tener cuidado al reasignar los nombres de las constantes o funciones a otros valores.

In []:

```
# Importando solo cos y pi desde matemáticas.
# No tenemos que usar "matemáticas" delante de cos o pi
from math import cos, pi
print(cos(pi))

# Sin embargo, tenemos que usarlo delante de otras funciones matemáticas.
math.log(pi)
```

O podemos importar cada función y valor de todo el módulo.

In []:

```
# Por último, podemos importar todo desde matemáticas usando el *
# Una vez más, no tenemos que usar "matemáticas". antemano
from math import *
log(pi)
```

No se preocupe demasiado por qué tipo de importación utilizar. A menudo es una elección de estilo de codificación que queda en manos de cada programador. En este curso, siempre importará los módulos necesarios cuando ejecute la celda de configuración (como la primera celda de código en esta práctica de laboratorio).

¡Pasemos a practicar algunas de las operaciones en tablas que aprendiste en las clases!

3. Operaciones con tablas.

La tabla `Lab02_poblacion_proyeccion_201.csv` contiene datos sobre el estimado de la población de las ciudades de Bogotá y Medellín desde 2018-2035 a nivel de comunas y corregimientos, estimados a junio 30 de cada año, con última actualización al 2024-07-25 (según los [datos abiertos de medellin \(https://www.medellin.gov.co/geomedellin/datosAbiertos/263\)](https://www.medellin.gov.co/geomedellin/datosAbiertos/263) y [dane \(https://www.dane.gov.co/index.php/estadisticas-por-tema/demografia-y-poblacion/proyecciones-de-poblacion/proyecciones-de-poblacion-bogota\)](https://www.dane.gov.co/index.php/estadisticas-por-tema/demografia-y-poblacion/proyecciones-de-poblacion/proyecciones-de-poblacion-bogota))

Ejecute la siguiente celda para cargar la tabla `poblacion`. No habrá resultados; no se espera ningún resultado ya que la celda contiene una declaración de asignación. Una declaración de asignación no produce ningún resultado (no produce ningún valor).

In []:

```
# Simplemente ejecuta esta celda
poblacion = Table.read_table(f'{path}/Dataset/Lab02_poblacion_proyeccion_201.csv')
```

Examinemos nuestra tabla para ver qué datos contiene.

Pregunta 3.1. Utilice el método `show` para mostrar las primeras 5 filas de `poblacion`.

Nota: Los términos "método" y "función" técnicamente no son lo mismo, pero para los propósitos de este curso, los usaremos indistintamente.

Pista: `tbl.show(3)` mostrará las primeras 3 filas de la tabla denominada `tbl`. Además, asegúrese de no llamar a `.show()` sin un argumento, ya que esto bloqueará su kernel.

In []:

```
poblacion.num_columns
```

Observe que faltan algunos de los valores en esta tabla, como se indica con "nan". Esto significa que el valor no está disponible o no es aplicable (como es el caso de las últimas columnas de la tabla), eso no significa que todas las columnas tengan valores "nan". ¡Note también la cantidad de filas y columna en el `dataset` !

Número de columnas: `num_columns`

La propiedad de la tabla `num_columns` devuelve el número de columnas de una tabla. (Una "propiedad" es solo un método al que no es necesario llamar agregando paréntesis).

Llamada de ejemplo: `tbl.num_columns` devolverá el número de columnas en una tabla llamada `tbl`

Pregunta 3.2. Utilice `num_columns` para encontrar el número de columnas en nuestro conjunto de datos de mercados de agricultores.

Asigne el número de columnas a `poblacion_columns`.

In []:

```
# BEGIN SOLUTION NO PROMPT
poblacion_columns = poblacion.num_columns

# END SOLUTION

""" # BEGIN PROMPT
poblacion_columns= ...
print("La tabla tiene ", poblacion_columns, "columnas!")
""" # END PROMPT
```

In []:

```
grader.check("q32")
```

Número de filas: `num_rows`

De manera similar, la propiedad `num_rows` te indica cuántas filas hay en una tabla.

In []:

```
# Ejecuta la siguiente celda
poblacion_rows = poblacion.num_rows
print("la tabla tiene", poblacion_rows, "filas!")
```

La tabla anterior solo tiene 41 filas o registros, sin embargo, el potencial de trabajar desde python es que podemos procesar muchos más datos como lo veremos más adelante en el curso. la cantidad de datos que podemos procesar, depende de nuestra capacidad de memoria y disco duro, tu servidor actual cuenta con: 2Gb Ram y 10 Gb de disco duro.

Nota : Es importante que poco a poco, se vaya familiarizando con las palabras del inglés, por ejemplo, rows y columns traducen como filas y columnas. Toda la sintaxis de python y sus librerías debe ser escrito en el lenguaje estricto de python, a diferencia de las variables o expresiones que tu puedes crear con el nombre que desees.

In []:

```
poblacion
```

select

La mayoría de las columnas tratan sobre las poblaciones de la ciudad de Medellín y Bogotá en un año específico, con información sobre las coordenadas y nombre de la comuna o localidad. Si no estamos interesados en todos los detalles, la tabla puede volverse difícil de leer.

En tales situaciones, podemos usar el método de tabla `select` para elegir solo las columnas que queremos en una tabla en particular. Este método toma varios argumentos, y cada uno debe ser el nombre de una columna, el cual devuelve una nueva tabla con solo esas columnas que se presentan en el orden en el que fueron enumeradas.

Por ejemplo, si queremos seleccionar solo el nombre de la comuna o localidad, las coordenadas de latitud, longitud y la población en el año 2018 de nuestra tabla `poblacion`, usaríamos el siguiente código:

```
poblacion_2018 = data[['LOCALIDAD', 'LATITUD', 'LONGITUD', '2018']]
```

Pregunta 3.3. Utilice `select` para crear una tabla con solo el LOCALIDAD, CIUDAD, LATITUD, y LONGITUD. Llame a esa nueva tabla "poblacion_pos_2018". (La latitud y la longitud hacen referencia a las [coordenadas geográficas](https://es.wikipedia.org/wiki/Coordenadas_geogr%C3%A1ficas#/media/Archivo:Latitud_y_Longitud_en_la_Tierra.svg) (https://es.wikipedia.org/wiki/Coordenadas_geogr%C3%A1ficas#/media/Archivo:Latitud_y_Longitud_en_la_Tierra.svg) de las comunas)

Sugerencia: Asegúrese de ser exacto al usar nombres de columnas con `select`; ¡Compruebe la escritura de las mayúsculas hasta dos o tres veces!

In []:

```
import numpy as np
from datascience import *
poblacion = Table.read_table(f'{path}/Dataset/Lab02_poblacion_proyeccion_201.csv')

# BEGIN SOLUTION NO PROMPT
poblacion_2018=poblacion.select('LOCALIDAD','Ciudad', 'LATITUD', 'LONGITUD', '2018')
# END SOLUTION

""" # BEGIN PROMPT
poblacion_pos_2018= ...
""" # END PROMPT
poblacion_2018
```

In []:

```
grader.check("q33")
```

drop

`drop` tiene el mismo propósito que `select`, pero elimina las columnas que usted proporciona en lugar de las que no proporciona. Al igual que "seleccionar", "soltar" devuelve una nueva tabla.

Pregunta 3.4. Supongamos que simplemente no desea las columnas `Unnamed: 0` y `2031` en `poblacion`. Cree una tabla que sea una copia de `poblacion` pero que no incluya esas columnas. Llame a esa tabla `poblacion_sin_2031`.

In []:

```
import numpy as np
from datascience import *
poblacion = Table.read_table(f'{path}/Dataset/Lab02_poblacion_proyeccion_201.csv')

# BEGIN SOLUTION NO PROMPT
poblacion_sin_2031 = poblacion.drop("Unnamed: 0", "2031")

# END SOLUTION

""" # BEGIN PROMPT
poblacion_sin_2031 = ...
""" # END PROMPT
poblacion_sin_2031
```

In []:

```
grader.check("q34")
```

Ahora supongamos que queremos responder algunas preguntas sobre la poblacion. Por ejemplo, ¿qué comuna o localidad entre Medellín y Bogotá tuvo la mayor población en el 2018?

Para responder a esto, ordenaremos `poblacion_2018` en el año 2018.

In []:

```
poblacion_2018.sort('2018')
```

Lo anterior no respondió a nuestra pregunta porque ordenamos de menor a mayor la columna 2018. Para ver las longitudes más grandes, tendremos que ordenar en orden inverso.

In []:

```
poblacion_2018.sort('2018', descending=True)
```

(El valor de `descending=True` se llama *argumento opcional*. Tiene un valor predeterminado(`default`) de `False` , cuando se hace explicito en la función `descending=True` , la función ordenará en orden descendente).

sort (ordenamiento)

Algunos detalles sobre el método `sort` :

1. El primer argumento para realizar el ordenamiento "sort" es el nombre de una columna por la que ordenar.
2. Si la columna tiene texto, "sort" ordenará alfabéticamente; si la columna tiene números, se ordenará numéricamente, ambos en orden ascendente de forma predeterminada.
3. El valor de `poblacion_2018.sort("2018")` es una *copia* de `poblacion_2018`; la tabla `poblacion_2018` no se modifica. Por ejemplo, si llamamos a `poblacion_2018.sort("x")` , ejecutar `poblacion_2018` por sí solo aún devolvería la tabla sin clasificar.
4. Las filas siempre permanecen juntas cuando se ordena una tabla. No tendría sentido ordenar solo una columna y dejar las otras columnas como están. Por ejemplo, en este caso, si ordenáramos solo la columna "2018", todos las poblaciones terminarían con las nombres del barrio incorrectas.

Pregunta 3.5. Cree una versión de `poblacion_2018` que esté ordenada por la coordenada (**latitud**), con las latitudes más grandes primero. Llámelo `poblacion_2018_por_latitud`.

In []:

```
import numpy as np
from datascience import *

poblacion = Table.read_table(f'{path}/Dataset/Lab02_poblacion_proyeccion_201.csv')

# BEGIN SOLUTION NO PROMPT
poblacion_sin_2031 = poblacion.drop("Unnamed: 0", "2031")
poblacion_loc_2018 = poblacion_sin_2031.select('LOCALIDAD', 'Ciudad', 'LATITUD', 'LONGITUD', '2018')

poblacion_2018_por_latitud = poblacion_loc_2018.sort('LATITUD', descending=True)
# END SOLUTION

""" # BEGIN PROMPT
poblacion_2018_por_latitud= ...
""" # END PROMPT

poblacion_2018_por_latitud
```

In []:

```
grader.check("q35")
```

Ahora queremos una tabla de todas las poblaciones solo de Medellín. ¿Cómo lo podemos hacer?, vamos a usar el método `where`.

In []:

```
poblacion_bogota = poblacion_2018_por_latitud.where('Ciudad', are.equal_to('Bogotá'))
poblacion_bogota
```

Ignora la sintaxis por el momento. En su lugar, intenta leer esa línea así:

```
Asigne el nombre poblacion_medellin a una tabla cuyas filas son las filas de la tabla
poblacion_2018_por_latitud donde la "Ciudad" es igual a Medellín ( are equal to "Bogotá" )
```

where

Ahora profundicemos un poco más en los detalles. `where` toma 2 argumentos:

1. El nombre de una columna. `where` busca filas donde los valores de esa columna cumplen algún criterio.
2. Un predicado que describe el criterio que debe cumplir la columna.

El predicado en el ejemplo anterior llamó a la función `are.equal_to` con el valor que queríamos, 'Bogotá'. Pronto veremos otros predicados.

`where` devuelve una tabla que es una copia de la tabla original, pero **con solo las filas que cumplen el predicado dado**.

Pregunta 3.6. Utilice la tabla `poblacion` para crear una tabla llamada `poblacion_medellin` que contenga la población de Medellín, ordenada de menor a mayor en año 2018

In []:

```
# BEGIN SOLUTION NO PROMPT
poblacion_medellin=poblacion_2018_por_latitud.where('Ciudad', are.equal_to('Medellín'))

# END SOLUTION

""" # BEGIN PROMPT
poblacion_medellin= ...
""" # END PROMPT

poblacion_medellin
```

```
In [ ]:
```

```
grader.check("q36")
```

Hasta ahora solo hemos estado usando `where` con el criterio o condición que requiere encontrar que los valores en una columna sean *exactamente* iguales a un cierto valor. Sin embargo, hay muchos otros criterios. Hemos enumerado algunos de los más comunes a continuación, pero si desea una lista más completa, consulte la sección de predicados en la [hoja de referencia de Python](https://www.data8.org/fa23/reference/#tablewhere-predicates) (<https://www.data8.org/fa23/reference/#tablewhere-predicates>).

Predicado	Ejemplo	Resultado
<code>are.equal_to</code>	<code>are.equal_to(50)</code>	Encuentra las filas con valores iguales a 50
<code>are.not_equal_to</code>	<code>are.not_equal_to(50)</code>	Encuentra las filas con valores diferentes a 50
<code>are.above</code>	<code>are.above(50)</code>	Encuentra las filas con valores mayores (y no iguales) a 50
<code>are.above_or_equal_to</code>	<code>are.above_or_equal_to(50)</code>	Encuentra las filas con valores mayores o iguales a 50
<code>are.below</code>	<code>are.below(50)</code>	Encuentra las filas con valores menores a 50
<code>are.between</code>	<code>are.between(2, 10)</code>	Encuentra las filas con valores mayores o iguales a 2 y menores a 10
<code>are.between_or_equal_to</code>	<code>are.between_or_equal_to(2, 10)</code>	Encuentra las filas con valores mayores o iguales a 2 y menores o iguales a 10

4. Analizando un conjunto de datos

Ahora que está familiarizado con las operaciones de tablas, ¡respondamos una pregunta interesante sobre un conjunto de datos!

Ejecute la siguiente celda para cargar la tabla `imdb`. Contiene información sobre las 250 películas mejor calificadas en IMDb.

```
In [ ]:
```

```
# Simplemente ejecuta esta celda
imdb = Table.read_table('imdb.csv')
imdb
```

A menudo, queremos realizar múltiples operaciones (clasificar, filtrar u otras) para convertir una tabla que tenemos en algo más útil. Puede realizar estas operaciones una por una, por ejemplo.

```
first_step = original_tbl.where("col1", are.equal_to(12))
second_step = first_step.sort('col2', descending=True)
```

Sin embargo, dado que el valor de la expresión `original_tbl.where("col1", are.equal_to(12))` es en sí mismo una tabla, puedes simplemente llamar a un método de tabla en él:

```
original_tbl.where("col1", are.equal_to(12)).sort('col2', descending=True)
```

Debe organizar su trabajo de la manera que tenga más sentido para usted, utilizando nombres informativos para las tablas intermedias que cree.

Pregunta 4.1. Cree una tabla de películas estrenadas entre 2010 y 2015 (inclusive) con calificaciones superiores a 8. La tabla solo debe contener las columnas `Title` and `Rating`, **en ese orden**.

Asigne la tabla al nombre `above_eight`.

Pista: Piensa en los pasos que debes seguir e intenta ordenarlos en un orden que tenga sentido. ¡Siéntete libre de crear tablas intermedias para cada paso, pero asegúrate de asignar a tu tabla final el nombre "above_eight"!

In []:

```
import numpy as np
from datascience import *

imdb = Table.read_table('imdb.csv')

# BEGIN SOLUTION NO PROMPT

movies_between_2010_2015 = imdb.where('Year', are.between_or_equal_to(2010, 2015))
high_rated_movies = movies_between_2010_2015.where('Rating', are.above(8))
above_eight = high_rated_movies.select('Title', 'Rating')

# END SOLUTION

""" # BEGIN PROMPT
...
...
above_eight= ...
""" # END PROMPT

above_eight
```

In []:

```
grader.check("q41")
```

Pregunta 4.2. Utilice el método `num_rows` (y aritmética) para encontrar la *proporción* de películas en el `dataset` que se estrenaron entre 1900 y 1999, y la *proporción* de películas en el `dataset` que se estrenaron en el año 2000 ó después.

Asigne una tabla "proporcion_en_siglo_20" a el número de películas en el `dataset` que se estrenaron entre 1900 y 1999, y "proporcion_en_siglo_21" a el número de películas en el `dataset` que se estrenaron en el año 2000 o después.

Pista: La *proporción* de películas estrenadas en el siglo XX es la *proporción* de películas estrenadas en el siglo XX, dividida por el *número total* de películas.

In []:

```
import numpy as np
from datascience import *

imdb = Table.read_table('imdb.csv')

# BEGIN SOLUTION NO PROMPT

num_peliculas_en_dataset = imdb.num_rows
num_en_siglo_20 = imdb.where('Year', are.between_or_equal_to(1900, 1999))
num_en_siglo_21 = imdb.where('Year', are.above_or_equal_to(2000))

proporcion_siglo_20 = num_en_siglo_20.num_rows/num_peliculas_en_dataset
proporcion_siglo_21 = num_en_siglo_21.num_rows/num_peliculas_en_dataset

# END SOLUTION

""" # BEGIN PROMPT
num_peliculas_en_dataset = ...
num_en_siglo_20 = ...
num_en_siglo_21 = ...
proporcion_siglo_20 = ...
proporcion_siglo_21 = ...
""" # END PROMPT

print("Proporcion en el siglo 20 :", proporcion_siglo_20)
print("Proporcion en el siglo 21 :", proporcion_siglo_21)
```

In []:

```
grader.check("q42")
```

5. Resumen

Para su referencia, aquí hay una tabla de todas las funciones y métodos que vimos en esta práctica de laboratorio. ¡Aprenderemos más métodos para agregar a esta tabla la próxima semana!

Nombre	Ejemplo	Propósito
<code>sort</code>	<code>tbl.sort("N")</code>	Crear una copia de una tabla ordenada por los valores de una columna
<code>where</code>	<code>tbl.where("N", are.above(2))</code>	Crea una copia de una tabla con solo las filas que coinciden con algún <i>predicado</i>
<code>num_rows</code>	<code>tbl.num_rows</code>	Calcular el número de filas en una tabla
<code>num_columns</code>	<code>tbl.num_columns</code>	Calcular el número de columnas en una tabla
<code>select</code>	<code>tbl.select("N")</code>	Crear una copia de una tabla con solo algunas de las columnas
<code>drop</code>	<code>tbl.drop("N")</code>	Crear una copia de una tabla sin algunas de las columnas

¡Lego quiere felicitarte por terminar el Laboratorio 2!



¡Terminaste con el laboratorio!

Información importante sobre el envío:

- **Ejecuta todas las pruebas** y verifica que todas pasan
- **Guardar** desde el menú **Archivo**
- **Ejecute la celda final para generar el archivo zip**
- **Haga clic en el enlace para descargar el archivo zip**
- Luego, ve a [Gradescope \(https://www.gradescope.com/courses/576157\)](https://www.gradescope.com/courses/576157) y envía el archivo zip a la tarea correspondiente. El nombre de esta tarea es "Lab XX Autograder", donde XX es el número de laboratorio: 01, 02, 03, etc.
- Si terminas temprano en el laboratorio regular, **pídele a uno de los miembros del personal que te marque**.

Es su responsabilidad asegurarse de que su trabajo esté guardado antes de ejecutar la última celda.

Referencia : Este material esta basado en [Data 8: The Foundations of Data Science \(https://www.data8.org/\)](https://www.data8.org/) - University of California, Berkeley.

Envío

Asegúrese de haber ejecutado todas las celdas de su cuaderno en orden antes de ejecutar la siguiente celda, para que todas las imágenes/gráficos aparezcan en el resultado. La siguiente celda generará un archivo zip para que lo envíe. **¡Guarde antes de exportar!**

In []:

```
# Save your notebook first, then run this cell to export your submission.  
#grader.export(pdf=False, run_tests=True)
```

Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

These are some submission instructions.

In []:

```
# Save your notebook first, then run this cell to export your submission.  
grader.export(run_tests=True)
```